



Technische Universität Berlin
Fakultät IV - Elektrotechnik und Informatik

STUDIERENDEN KOMPASS

"NAVIS"

Dokumentation

im Modul

Projekt Medienerstellung

im Sommersemester 2026

Autoren: Alessio Beulcke
Lena Bohlje
Jacob Matteo Rene Eckstein
Anna Glavatska
Jasmin Heiße
Volodymyr Honcharov
Lennart Nicolas Lunt
Emma Weiland

Supervisorin: Judith Maria Bucher

Eingereicht am: Freitag, den 07. August 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Umsetzung	4
2	Komponenten	6
2.1	Footer	6
2.2	modul-card	6
2.3	modul-custom	6
2.4	semester-card	7
2.5	semester-modul-card	7
2.6	nav-bar	7
2.7	modul-feedback	8
2.8	check-modules	9
3	Frontend	10
3.1	Responsive Design	10
3.2	Theme	10
3.3	Planer	10
3.4	Module	11
3.5	Technische Architektur und Datenabruf der Modulsuche	12
3.5.1	Graphen-Traversierung (ladeModulBasisAction)	12
3.5.2	On-Demand-Loading und Dispatching-Logik (ladeDetailedModulAction)	13
3.5.3	Batch-Auflösung (ladeModulBasisByIdsAction)	13
3.5.4	Fuzzy-Matching (ModulsucheWrapper und Fuse.js)	14
3.6	Dashboard	14
3.7	Einstellungen	16
4	Backend	17
4.1	MOSES-API Endpoints und Modulfetch	17
4.2	Asynchrone ETL-Pipeline und Defensive Programming	18
4.3	NLP-Pipeline und Vektor-basiertes Keyword-Matching	19
4.4	Backend bei TypeScript	20
4.5	Automation	21
5	Supabase	24
5.1	Buckets	24
5.2	Authentifizierung und Sicherheit	27
5.3	Planer und Relationale Tabellen	28
5.4	Debugging und Realtime Logging	30

6	Vercel	31
6.1	Hosting: Next.js App auf Vercel	31
6.2	Serverless Vercel	31
6.3	Progressive Web App	31
7	Git	33
7.1	Workflows und CI/CD	33
7.2	DevOps und Branchmanagement	33
7.3	Ausblick	34
	Abbildungsverzeichnis	35
	Codelistenverzeichnis	36
	Quellenverzeichnis	37

1 Einleitung

Lena Bohlje

Mit unserem Projekt „NAVIS“ (ehemals Studierenden Kompass) wurde ein interaktives System zur individuellen Studienplanung entwickelt. NAVIS soll Studierenden eine bessere Strukturierung ihres Studienverlaufs ermöglichen und bestehende Systeme sinnvoll ergänzen, ohne diese zu ersetzen. Im Fokus stehen dabei:

- Zentrale Übersicht: Darstellung der zu belegenden und wählbaren Module
- Semesterplanung: Flexibles Zusammenstellen einzelner Semester
- Individueller Studienverlauf: Visualisierung des eigenen Plans basierend auf persönlichen Interessen, Stärken oder beruflichen Zielen
- Orientierung bei Wahlpflichtmodulen: Unterstützung bei der Modulauswahl

Zudem soll NAVIS den Austausch unter den Studierenden fördern. Durch geplante Bewertungs- und Kommentarfunktionen werden persönliche Erfahrungen sichtbar gemacht, um zukünftigen Studierenden die Entscheidungsfindung zu erleichtern.

1.1 Umsetzung

Lena Bohlje

NAVIS ist eine moderne Web-App, die sowohl auf Desktop-Geräten als auch auf mobilen Endgeräten vollumfänglich nutzbar ist. Für die Realisierung kommen zeitgemäße Frontend- und Backend-Technologien zum Einsatz. Die Backend-Infrastruktur sowie das Authentifizierungs- und Login-System werden mit Supabase¹ umgesetzt, um eine sichere Datenhaltung zu gewährleisten. Die Server- und Client-Logik sowie die API-Anbindungen werden über Next.js², React³ und TypeScript⁴ realisiert, während das Hosting und Deployment der Plattform über Vercel⁵ erfolgt. Eine Besonderheit des Projekts liegt in der direkten Anbindung an die MOSES-Datenbank der TU Berlin. Die technische Umsetzung im Frontend erfolgt schrittweise mittels Tailwind CSS⁶ und Next.js, wobei eine intuitive und übersichtliche Nutzerführung im Vordergrund steht. Da zentrale

¹siehe [Supabase]

²siehe [Nextjs]

³siehe [React]

⁴siehe [TypeScript]

⁵siehe [Vercel]

⁶siehe [TailwindCSS]

Funktionen auf Daten von MOSES⁷ basieren, müssen potenzielle technische und organisatorische Einschränkungen kontinuierlich berücksichtigt werden.

⁷siehe [MOSES]

2 Komponenten

Dieser Abschnitt beschreibt die wichtigsten Komponenten, deren Verständnis für die folgenden Abschnitte vorausgesetzt wird.

2.1 Footer

Lena Bohlje

Neben dem Logo von NAVIS und einer kurzen Beschreibung befinden sich hier die Buttons zum Einloggen und Registrieren. Außerdem wird auf dieser Seite unser Video aus der Zwischenpräsentation angezeigt, in dem NAVIS anschaulich und vollständig vorgestellt wird. Darüber hinaus sind ein „About Us“-Bereich mit den Credits sowie die Datenschutzhinweise und die AGB zu finden.

2.2 modul-card

Lena Bohlje

Die Modulkarten im Modulkatalog „Module“ enthalten die Daten aus der MOSES-Datenbank. Nachdem man einen Filter ausgewählt oder etwas in die Suchleiste eingegeben hat, werden die Modulnamen sowie Informationen dazu angezeigt, ob das Modul bereits abgeschlossen wurde, wie viele ECTS es umfasst, in welchem Turnus es angeboten wird und welchem Bereich es im jeweiligen Studiengang zugeordnet ist.

Klappt man ein Modul auf, werden weitere Informationen angezeigt, beispielsweise die Lernergebnisse, die Prüfungsform, ob das Modul benotet ist, ob Voraussetzungen für die Belegung bestehen, ein Link zur originalen MOSES-Seite, die Möglichkeit, das Modul einem Semester im Planer hinzuzufügen, sowie das Studierendenfeedback.

2.3 modul-custom

Jasmin Heiße

Das Custom-Modul wird über den Button modul-custom-button auf der modules page geöffnet. Studenten können benutzerdefinierte Module anlegen, die dann mit den Eingaben des Studenten in der Datenbank gespeichert werden. Mit den Informationen können dann ganz normal die Modulkarten geladen werden.

Um Jobs besser eintragen zu können, gibt es einen Toggle, der die geforderten Eingaben verändert. Die eingetragenen Daten werden in der normalen Modultabelle gespeichert.

2.4 semester-card

Jasmin Heiße

Jede Semester-Card stellt ein Semester dar. Es wird angezeigt das wievielte Semester es ist, sowie die Summe der ECTS der Module, die im passenden Semester gespeichert sind und die Anzahl der enthaltenen Module. Außerdem gibt es einen Button "Module hinzufügen", der auf die modules page zur Modulsuche führt.

2.5 semester-modul-card

Lena Bohlje

Hierbei handelt es sich um die Module, die dem Planer und somit einem bestimmten Semester zugeordnet wurden. In der eingeklappten Ansicht werden der Modulname, die Anzahl der ECTS, der Turnus sowie ein Link zur originalen MOSES-Seite angezeigt. Außerdem kann das Modul über das Mülleimersymbol aus dem Planer entfernt werden. Sofern bereits eine Note eingetragen wurde, wird diese ebenfalls in der eingeklappten Ansicht angezeigt.

Klappt man das Modul auf, werden weitere Informationen sichtbar. Dazu gehören der zugehörige Bereich, die Beschreibung, der Arbeitsaufwand – wahlweise als Gesamtstunden im Semester oder als Stunden pro Woche – sowie der Turnus und die Prüfungsform. Zusätzlich kann die Anzahl der benötigten Prüfungsversuche eingetragen, eine Note vergeben und festgelegt werden, ob diese regulär gewichtet wird oder nicht in die Gesamtnote einfließt. Im unteren Bereich besteht außerdem die Möglichkeit, das Modul als abgeschlossen zu markieren oder direkt zum Bewertungsforum weitergeleitet zu werden.

2.6 nav-bar

Lennart Nicolas Lunt

Die Navigationsleiste ist in allen geschützten Bereichen der Anwendung eingebunden und passt sich der Bildschirmgröße an. Auf Desktop-Geräten liegt sie als feste Seitenleiste über die gesamte Bildschirmhöhe am linken Rand. Auf Smartphones und Tablets wird sie stattdessen zu einer schmalen Leiste am oberen Rand, die sich über ein Menüsymbol zu einer bildschirmfüllenden Ansicht aufklappen lässt und über ein Kreuz wieder geschlossen wird. Solange das mobile Menü geöffnet ist, wird das Scrollen der dahinterliegenden Seite unterbunden, damit sich beim Bedienen des Menüs nicht ungewollt der Seiteninhalt mitbewegt.

Am oberen Rand befindet sich das Logo von NAVIS, das zugleich als Verknüpfung zum Planer dient. Logo und Schriftzug liegen in je einer Fassung für den hellen und den dunklen Modus vor und werden passend zum aktiven Theme eingeblendet. Darunter folgen die Navigationspunkte, die zentral in den Konstanten hinterlegt sind und jeweils aus einem Symbol und einer Bezeichnung bestehen. Der aktuell geöffnete Bereich wird farblich hervorgehoben; dabei werden auch Unterseiten berücksichtigt, sodass die Hervorhebung beispielsweise bei den Einstellungen und deren Unterseiten erhalten bleibt. Auf mobilen Geräten schließt sich das Menü nach der Auswahl eines Navigationspunktes automatisch.

Im unteren Bereich werden der Benutzername und der gewählte Studiengang angezeigt, die aus der Profiltable in Supabase geladen werden. Daneben steht das Profilbild; ist keines hinterlegt, wird stattdessen ein aus den Initialen erzeugter Avatar dargestellt. Damit Änderungen am Profil unmittelbar sichtbar werden, reagiert die Navigationsleiste auf entsprechende Ereignisse aus den Einstellungen und lädt die Profildaten dann erneut. Auf diese Weise erscheinen ein neu hochgeladenes Profilbild oder ein geänderter Studiengang sofort, ohne dass die Seite neu geladen werden muss. Abgeschlossen wird die Leiste durch die Umschaltung zwischen hellem und dunklem Modus sowie die Schaltfläche zum Abmelden.

2.7 modul-feedback

Lennart Nicolas Lunt

Das Studierendenfeedback ermöglicht es, Module zu bewerten und die Erfahrungen anderer Studierender einzusehen. Der Bereich ist sowohl in den Modulkarten des Modulkatalogs als auch in den Modulkarten des Planers eingebunden und lässt sich dort über den Abschnitt „Studierenden-Feedback“ aufklappen.

Eine Bewertung besteht aus fünf Kategorien, die jeweils mit einem bis fünf Sternen bewertet werden: Dozent, Vorlesung, Tutorium, Aufwand und Organisation. Diese Aufteilung wurde bewusst einer einzelnen Gesamtnote vorgezogen, da ein Modul beispielsweise eine sehr gute Vorlesung, aber ein schlecht organisiertes Tutorium haben kann. Ergänzt wird jede Bewertung um das Semester, in dem das Modul besucht wurde, sowie um einen Kommentar von bis zu 500 Zeichen. Abgeschickt werden kann eine Bewertung erst, wenn alle fünf Kategorien bewertet und ein Kommentar verfasst wurde. Angezeigt wird jede Bewertung anschließend mit dem Benutzernamen, einem aus den Initialen erzeugten Avatar, dem Semester und dem Erstellungsdatum.

Oberhalb der einzelnen Bewertungen fasst eine Übersicht alle Rückmeldungen zusammen: die Gesamtnote als Durchschnitt sämtlicher Bewertungen, die Verteilung auf die Stufen von einem bis fünf Sternen als Balkendiagramm sowie der Durchschnitt je Kategorie. Die

Sterne werden dabei auch anteilig dargestellt, sodass ein Durchschnitt von beispielsweise 4,2 als vier volle und ein zu 20 Prozent gefüllter Stern erscheint. Die Liste der Bewertungen lässt sich nach „Hilfreichste“, „Neueste“ und „Beste“ sortieren; die Sortierung erfolgt dabei im Browser, ohne dass die Daten erneut geladen werden.

Gespeichert werden die Bewertungen in der Supabase-Tabelle **bewertung**. Der Zugriffsschutz ist über Row Level Security direkt in der Datenbank hinterlegt: Angemeldete Nutzer können alle Bewertungen lesen, aber ausschließlich eigene anlegen und löschen. Eine Prüfung allein im Anwendungscode wäre an dieser Stelle wirkungslos, da Anfragen an Supabase auch unter Umgehung der Benutzeroberfläche gestellt werden können. Die eigene Bewertung ist an einem Löschen-Button erkennbar, der bei fremden Bewertungen nicht angezeigt wird. Aus Gründen der Ladezeit werden die Bewertungen eines Moduls zudem erst beim Aufklappen abgerufen und nicht bereits beim Anzeigen der Modulkarte.

2.8 check-modules

Anna Glavatska Um sicherzustellen, dass im Planer hinterlegte Module dem aktuellen Systemstand entsprechen, gleicht eine Aktualitätsprüfung die gespeicherten Daten mit der MOSES-API ab. Über die in Supabase ergänzte Spalte **module_type** wird zwischen selbst erstellten Inhalten (**custom**, stets aktuell) sowie der Standard- (**basic**) und erweiterten Suche (**extended**) differenziert. Beide Wege führen in einen gemeinsamen Prüfpfad über den Endpunkt `/bolognamodulversion/version_id`, der Modulname, ECTS und das Feld **semesterbis** abgleicht und bei Abweichungen oder veralteten Ständen entsprechende Warnungen ausgibt. Dabei wird bei den Basis-Modulen mit der Nutzergewählten StuPO abhängigen Version und bei den Extended-Modulen mit der maximalen (neusten) Version verglichen, da dort die Speicherung StuPO- und Studiengang-unabhängig ist (Wahl dieser Module ist voraussichtlich für den freien Wahlbereich oder aus Interesse einem Studiengangswechsel)

Die visuelle Umsetzung erfolgt über die Client-Komponente **check_modules**, welche die unterschiedlichen Stati (**UP_TO_DATE**, **WARNING**, **ERROR**) über eine zentrale Konfiguration (**STATUS_CONFIG**) mit Icons aus **lucide-react** abbildet. Gesteuert über einen **useState**-Hook für Hover-, Fokus- und Klick-Ereignisse, bietet die Komponente einen interaktiven und barrierefreien Tooltip zur genauen Fehler- oder Änderungsanzeige.

3 Frontend

3.1 Responsive Design

Lena Bohlje

Unsere Webseite ist auf Smartphones, Tablets und Laptops gut bedienbar und passt sich automatisch an die jeweilige Bildschirmgröße an. Die responsive Darstellung wurde mithilfe von Tailwind CSS umgesetzt.

3.2 Theme

Jasmin Heiße

Die Seite hat ein Farbschema, das auf den offiziellen Farben der TU Berlin basiert. Diese sind in globals.css definiert.

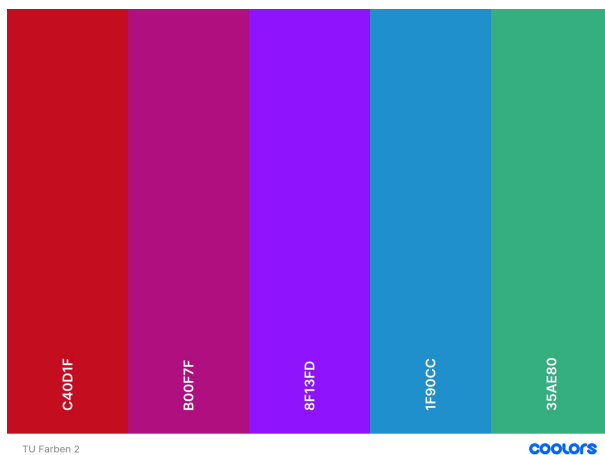


Abbildung 1: Farbschema erstellt mit coolers.

Die Farben für den Darkmode sind ebenfalls in globals.css definiert. Ergänzend ist der Darkmode manuell mit Tailwind CSS angepasst.

3.3 Planer

Lena Bohlje, Emma Weiland

„Plane dein Studium semesterweise“

Hier liegt der Kern von NAVIS: Hier lassen sich Module den einzelnen Semestern zuordnen und per Drag-and-Drop verschieben. Die Drag-and-Drop-Funktionalität wurde mit `dnd-kit`⁸ umgesetzt und die Implementierung umfasst:

⁸siehe [dnd-kit]

- **DndContext:** Das zentrale Steuerelement, das über `onDragStart` (Modul wird gegriffen) und `onDragEnd` (Modul wird abgelegt) die Benutzeroberfläche aktualisiert und die neue Position in der Datenbank speichert.
- **Droppables und Sortables:** Die Module dienen als verschiebbare Elemente (`useSortable`), während die Semester als Droptargets (`useDroppable`) fungieren.
- **DragOverlay:** Rendert eine schwebende Vorschau der Modulkarte, die dem Mauszeiger während des Ziehvorgangs folgt und so visuelles Feedback liefert.

Im Planer können sowohl ganze Semester als auch einzelne Module hinzugefügt oder gelöscht werden. Die Anzahl der Semester wird dabei automatisch mit der zu Beginn in den Einstellungen festgelegten Semesteranzahl synchronisiert und entsprechend aktualisiert.

Für jedes Semester werden die Anzahl der Module sowie die Summe der ECTS automatisch berechnet. Überschreitet ein Semester die empfohlene Grenze von 30 ECTS, wird eine Warnung angezeigt. Ebenso erscheint ein Hinweis, wenn ein Wintersemester-Modul (WiSe) einem Sommersemester (SoSe) zugeordnet wird oder umgekehrt.

3.4 Module

Lennart Nicolas Lunt

„Durchsuche und verwalte verfügbare Module.“

Der Modulkatalog listet alle Module des eigenen Studiengangs auf. Welcher Studiengang geladen wird, ergibt sich aus dem Benutzerprofil; ist dort noch kein Studiengang hinterlegt, erscheint statt der Liste ein Hinweis, diesen zunächst in den Einstellungen auszuwählen.

Die Moduldaten stammen aus der MOSES-Schnittstelle der TU Berlin. Der Abruf erfolgt über Server Actions und damit ausschließlich serverseitig. Auf diese Weise verlässt der API-Schlüssel den Server nicht, und es entstehen keine CORS-Probleme. Über drei Schaltflächen lässt sich zwischen allen Modulen, ausschließlich Pflichtmodulen und ausschließlich Wahlpflichtmodulen umschalten. Der erste Klick lädt die Module einmalig vollständig; jeder weitere Klick filtert nur noch den bereits geladenen Bestand, sodass keine erneute Abfrage nötig ist. Ergänzend kann über die Suchleiste nach Modulnamen gesucht werden. Da der Katalog eines Studiengangs mehrere hundert Module umfassen kann, werden bewusst nur Name, ECTS und Bereichspfad vorab geladen; alle weiteren Details folgen erst beim Aufklappen einer einzelnen Modulkarte.

Solange die Seite ihre Daten lädt, wird anstelle eines leeren Bereichs ein Skelett der späteren Seitenstruktur angezeigt. Über eine zusätzliche Schaltfläche lässt sich in die erweiterte

Suche wechseln. Module, die nicht im MOSES-Katalog enthalten sind, können über die Funktion „Eigenes Modul anlegen“ ergänzt werden (siehe Abschnitt 2.3).

Anna Glavatska

3.5 Technische Architektur und Datenabruf der Modulsuche

Die technische Realisierung der Modulsuche basiert auf einer mehrstufigen Abfragelogik über die MOSES-API der TU Berlin. Da das MOSES-Datenbankmodell viele Verzweigungen aufweist und Vercel-Serverless-Funktionen strengen Laufzeitbeschränkungen unterliegen, greift das Backend auf asynchrone Nebenläufigkeit, Batch-Verarbeitung und rekursive Datenauflösung zurück.

3.5.1 Graphen-Traversierung (ladeModulBasisAction)

Die Server Action `ladeModulBasisAction(studiengangId)` steuert die Abfrage des gesamten (Basis) Modulkatalogs. Da die MOSES-API keine flachen Modullisten pro Studiengang bereitstellt, erfolgt der Datenabruf als Graphen-Traversierung über verknüpfte Endpunkte:

```
/studiengang/{id}
└─ stupoList (Filter: profile.stupo_year oder max(id))
    └─ /studiengangsabbildung?stupoId={id}
        └─ modullisteList / bolognamodullisteList
            └─ [Pfad A: Bologna] -> /bolognamodulliste/{id} -> Gruppen -> Zuordnungen
            └─ [Pfad B: Standard] -> /modulliste/{id} -> Zuordnungen
```

- **StuPo-Auflösung und Fallback:** Das Studien- und Prüfungsordnungsjahr (`stupo_year`) des Nutzers wird aus der Tabelle `profiles` geladen, um in `stupoList` den passenden Eintrag zu finden. Wenn der Abgleich fehlschlägt oder kein Jahr hinterlegt ist, wird defensiv der Eintrag mit der höchsten numerischen ID (`max.id`) gewählt.
- **Pfad-Weiche und Deduplizierung:** Über die `studiengangsabbildung` wird ermittelt, ob das Bologna-Format (`isBologna`) vorliegt. Da Module in Gruppen mehrfach auftreten können, werden Referenz-IDs vor dem Abruf in einer Hash-Map (`Map<string, MosesRef>`) nach Modulnamen dedupliziert, wobei stets die höchste ID (`z.id > existing.id`) persistiert wird.

- **Asynchrone Batch-Verarbeitung:** Um das Verbindungslimit des MOSES-Servers nicht zu überlasten, wird das deduplizierte Array in Blöcke von 30 Einträgen (`BATCH_SIZE = 30`) aufgeteilt. Die Abfragen auf die Detail-Endpunkte werden pro Block parallel über `Promise.all` ausgeführt.
- **Rekursive Bereichspfad-Auflösung:** Für Nicht-Bologna-Module ermittelt die Hilfsfunktion `getBereichPfad(bereichId)` die Hierarchie der Studienbereiche (z. B. Pflichtbereich → Grundlagen → Mathematik) durch rekursives Traversieren des Attributs `parent.id`.

Das Resultat ist ein reduziertes Array vom Typ `ModulBasis[]` (`id`, `name`, `lp`, `bereichPfad`, `semester`).

3.5.2 On-Demand-Loading und Dispatching-Logik (`ladeDetailedModulAction`)

Detaillierte Modulattribute (z. B. Lernergebnisse, Voraussetzungen und Prüfungsformen) werden nach einem Lazy-Loading-Ansatz erst beim Aufklappen einer Modulkarte geladen.

- **Dispatcher-Funktion:** Die Server Action `ladeDetailedModulAction(modul_id, no_deg)` steuert als zentrale Vermittlungsschicht, ob ein Modul studiengangsgebunden (`ladeStandardDetails`) oder studiengangsunabhängig (`ladeExtendedDetails`) geladen wird.
- **Relationaler Abruf (`ladeStandardDetails`):** Zunächst wird die Studiengangs-zuordnung abgerufen (mit Fallback auf `/bolognamodullistenzuordnung/{id}`). Anschließend werden über die extrahierte `versionId` die Basis-, Beschreibungs- und Prüfungsdaten parallel via `Promise.all` geladen, im JSON-Format aggregiert und um eine offizielle MOSES-URL (`baueMosesLink`) ergänzt.
- **Studiengangsunabhängiger Abruf (`ladeExtendedDetails`):** Hier wird direkt auf `/bolognamodul/{id}` zugegriffen. Aus der `bolognamodulVersionList` wird die Version mit der höchsten ID (`max.id`) selektiert und über `semesterBis == null` auf Aktualität geprüft, bevor Beschreibung und Prüfungsmodalitäten parallel geladen werden.

3.5.3 Batch-Auflösung (`ladeModulBasisByIdsAction`)

Liefert ein Suchalgorithmus lediglich rohe Modul-IDs zurück (`{id, name, version_info}[]`), werden diese in 10er-Blöcken (`BATCH_SIZE = 10`) verarbeitet. Für jede ID erfolgt ein zweistufiger Fetch: Nach Ermittlung der aktuellsten Version aus `/bolognamodul/{id}` werden

ECTS-Anzahl (1p) und Turnus aus `/bolognamodulbeschreibung/{desc_id}` ausgelesen und als `ModulBasis`-Objekt formatiert. Fehlerhafte IDs werden im `catch`-Block abgefangen und am Ende über `.filter(Boolean)` verworfen.

3.5.4 Fuzzy-Matching (ModulsucheWrapper und Fuse.js)

Die clientseitige Steuerung der Sucharchitektur erfolgt über die Komponente `ModulsucheWrapper`, die mittels eines lokalen `useState`-Hooks zustandserhaltend zwischen der regulären MOSES-Suche und der erweiterten Suche umschaltet. Ein Wechsel des Suchmodus rendert ausschließlich den jeweiligen UI-Baum neu, ohne einen Seiten-Reload auszulösen.

Für die fehlerverzeihende Filterung der Module kommt die Bibliothek **Fuse.js** zum Einsatz, die als Datenquelle auf eine vorab aus Supabase geladene, strukturierte JSON-Datei (Siehe Abschnitt 4, Backend) zurückgreift. Der Algorithmus indiziert die darin enthaltenen Moduldatensätze im Browser und berechnet über eine Ähnlichkeitsmetrik einen Relevanz-Score für jede Nutzereingabe. Über einen definierten Schwellenwert (Threshold) erkennt die unscharfe Suche (Fuzzy-Matching) auch Tippfehler, Buchstabendreher oder unvollständige Suchbegriffe. Da der gesamte Abgleich direkt im Arbeitsspeicher auf der JSON-Struktur operiert, entfallen Latenzen durch wiederholte HTTP-Anfragen.

3.6 Dashboard

Jacob Matteo Rene Eckstein, Volodymyr Honcharov

„Verfolge deinen Weg zum Abschluss.“

Das Dashboard soll den Studierenden einen schnellen Überblick über ihren aktuellen Studienfortschritt geben und die wichtigsten Informationen übersichtlich auf einer Seite zusammenfassen.

Die benötigten Daten werden aus Supabase geladen und für die Darstellung des Gesamtfortschritts, der Leistungspunkte, des aktuellen Fachsemesters sowie des Notendurchschnitts verwendet. Das aktuelle Fachsemester wird aus den Benutzereinstellungen übernommen und dient als Grundlage für die Anzeige der Module des aktuellen und des nächsten Semesters.

Der Gesamtfortschritt wird anhand der bereits erreichten ECTS-Punkte im Verhältnis zu den insgesamt benötigten ECTS berechnet und sowohl grafisch als auch numerisch dargestellt. Der Notendurchschnitt wird ECTS-gewichtet aus allen abgeschlossenen und benoteten Modulen berechnet. Dabei werden ausschließlich Module berücksichtigt, die im

Planer als abgeschlossen markiert sind und für die eine Note eingetragen wurde. Zusätzlich werden die Anzahl der abgeschlossenen Module, der Fortschritt im aktuellen Semester sowie verschiedene Meilensteine angezeigt.

Außerdem wurde ein Einstellungsbereich integriert, über den einzelne Dashboard-Bereiche ein- oder ausgeblendet werden können. Die gewählten Anzeigeeinstellungen werden lokal im Browser gespeichert und bleiben dadurch auch nach einem erneuten Öffnen der Anwendung erhalten. Zusätzlich können dort die insgesamt benötigten ECTS sowie eine persönliche Wunschnote festgelegt werden, sodass die Berechnungen individuell angepasst werden können.

3.7 Einstellungen

Emma Weiland

„Verwalte dein Profil und Präferenzen.“

Die Einstellungskomponente dient der zentralen Anpassung und Kontrolle aller persönlichen Nutzerdaten und Systemeinstellungen. Sie gliedert sich in folgende Kernbereiche:

- **Profil:** Zeigt den Namen, die Email-Adresse und den aktuellen Studiengang. Nutzer können hier ein persönliches Profilbild hochladen. Wird kein Bild hinterlegt, generiert unser System automatisch einen Avatar basierend auf den eigenen Initialen (aus der angegebenen Email-Adresse)
- **Studiengang:** Der bei der Erstregistrierung gewählte Studiengang wird hier angezeigt und lässt sich bei Bedarf jederzeit anpassen.
- **Studien- und Prüfungsordnung:** Eine StuPo kann hier hinterlegt und jederzeit geändert werden.
- **Semesterplanung:** Hier werden die akademischen Eckdaten konfiguriert: darunter das aktuelle Fachsemester, der jeweilige Semesterturnus (Winter- oder Sommersemester) sowie die geplante Studiendauer.
- **Email und Passwort zurücksetzen:** Hier kann die hinterlegte Email-Adresse geändert und ein neues Passwort festgelegt werden.
- **Konto löschen:** Der Nutzer hat die Möglichkeit das eigene Benutzerkonto inklusive aller verknüpften Daten permanent zu löschen. Zur Sicherheit muss diese Aktion jedoch vorab bestätigt werden.
- **Admin Settings:** Ein Nutzer mit Admin Rechten kann hier auf die Datenbank Logs von Github-Actions zugreifen und Skripte triggern.

4 Backend

Anna Glavatska

Die Backend-Architektur umfasst die Kernlogik der Webanwendung sowie die API-Anbindungen an die MOSES-Datenbank, die mit Next.js, React und TypeScript implementiert wurden. Diese Architektur ermöglicht eine nahtlose Integration von serverseitigem Rendering und clientseitiger Interaktivität.

Bei der Planung der Umsetzung der erweiterten Modulsuche stellte sich jedoch heraus, dass der komplexe Datenabruf von MOSES eine sehr große Laufzeit aufweisen würde. Dies stellte eine Herausforderung dar, da Next.js auf Vercel nach der Serverless-Architektur arbeitet – das bedeutet, dass Code in isolierten, zustandslosen Funktionscontainern (Functions-as-a-Service) nur bei eingehenden Anfragen kurzzeitig ausgeführt wird. Aufgrund des kostenlosen Vercel-Tiers sind diese Ausführungszeiten (Execution Timeouts) streng auf maximal 10 Sekunden pro HTTP-Request limitiert. Zudem würde eine synchrone Ausführung eine unzumutbar lange Wartezeit für die Nutzer bedeuten. Würden mehrere Nutzer die erweiterte Suche an einem Tag durchführen, so müssten wir noch weitere Limitierungen in Betracht ziehen, besonders den Arbeitsspeicher (RAM), da die Aktion auf Web-Basis durchzuführen war.

Um serverseitige Timeouts zu verhindern und die Nutzerfreundlichkeit durch kurze Wartezeiten zu verbessern, wurde die Architektur um eine asynchrone, entkoppelte Python-ETL-Pipeline erweitert, die in einem Docker-Container⁹ über GitHub Actions¹⁰ ausgeführt wird.

4.1 MOSES-API Endpoints und Modulfetch

Der Datenabruf aus der MOSES-API basiert auf einer Breadth-First-Graphentraversierung, die in der Python-Pipeline asynchron implementiert wurde. Zunächst werden über den Endpunkt `/studiengang` alle verfügbaren Studiengänge seitenweise (*Pagination* mit 500er-Seiten) abgerufen.

Der Algorithmus prüft anschließend in der Funktion `fetch_is_Bologna` über den Endpunkt `/studiengangsabbildung`, ob ein Studiengang das Bologna-Format verwendet oder auf alte Modullisten zurückgreift. Daraus ergeben sich zwei Verarbeitungspfade:

⁹siehe [Docker]

¹⁰siehe [GitHubActions]

- **Bologna-Pfad (Path A):** Navigiert über `/bolognamodulliste/max_id` zur Ermittlung der Gruppen-IDs (`fetch_a_groups`), greift auf `/bolognamodullistengruppe` für die Zuordnungs-IDs zu (`fetch_a_zuordnungen`) und ermittelt über `/bolognamodullistenzuordnung` schließlich die Versions-ID (`fetch_a_version`).
- **Standard-Pfad (Path B):** Navigiert direkt über `/modulliste/max_id` zu den Zuordnungen (`fetch_b_zuordnungen`) und von dort zur Modulversion (`fetch_b_version`).

Beide Pfade mappen am Ende über `fetch_module_id` und den Endpunkt `/bolognamodulversion/vers_id` auf die finale Modul-ID.

Ein zentrales Element der Hilfsfunktion `fetch_single` ist die integrierte Deduplizierung, die sicherstellt, dass bei IDs von Listen stets der maximale beziehungsweise aktuellste Eintrag verwendet wird.

Zudem wird über den Endpunkt `/bolognamodulversion/version_id` der Parameter `semesterBis` ausgelesen, um in der Variable `version_info` zu kennzeichnen, ob ein Modul aktiv oder bereits veraltet ist.

4.2 Asynchrone ETL-Pipeline und Defensive Programming

Die Steuerung des gesamten Datenflusses, der Machine Learning Pipeline zur Keyword-Extraktion sowie des Uploads von realtime Logs zu Supabase erfolgt über das Skript `async_extended_search.py`.

- **Nebenläufigkeit mit `httpx` und `asyncio`:** Anstelle blockierender Standard-Requests setzt das Skript auf den asynchronen `httpx.AsyncClient`. Um den MOSES-Server vor Überlastung zu schützen, begrenzt ein explizites Semaphore (`asyncio.Semaphore(20)`) die maximal parallel aktiven HTTP-Anfragen auf 20 und einem Verbindungslimit von 50 Sockets.
- **Exponential-Backoff-Retry-Logik:** Die zentrale Wrapper-Funktion `fetch_with_retry` fängt Verbindungs-Timeouts sowie kritische HTTP-Statuscodes (429, 500, 502, 503, 504) ab. Tritt ein solcher Fehler auf, wird der Request mit einer exponentiell ansteigenden Wartezeit (`2 ** attempt` Sekunden) bis zu dreimal wiederholt. 404-Fehler werden kontrolliert abgefangen, um den ETL-Prozess nicht fehlschlagen zu lassen.
- **Fehlertoleranter Daten-Gathering-Prozess:** Komplexe Abfrage-Gruppen nutzen `asyncio.gather(*tasks, return_exceptions=True)`. Dadurch führt ein fehlerhafter API-Call bei einem einzelnen Modul nicht zum Abbruch der gesamten Pipeline. Stattdessen werden Ausnahmen im Terminal/Docker gelogged und die übrigen Datensätze fehlerfrei weiterverarbeitet.

4.3 NLP-Pipeline und Vektor-basiertes Keyword-Matching

Die Rohdaten werden anschließend in ein Dictionary folgender Form (pro Element) sortiert:

```
1 my_dict[module_id] = {  
2     "de_name": de_name,  
3     "en_name": en_name,  
4     "version": version_info,  
5     "studiengaenge": [],  
6     "lehrinhalt": Lehrinhalt,  
7     "words": []  
8 }
```

Listing 1: Struktur des Modul-Dictionaries in der ETL-Pipeline

Für jede `module_id` werden aus dem Lehrinhalt die Schlagwörter (`words`) mithilfe von Pytorch¹¹ extrahiert. Da diese Prozesse sehr rechenintensiv sind und die Event-Loop von `asyncio` blockieren würden, wird der gesamte NLP-Block über `asyncio.to_thread(find_internal_candidates)` in einen separaten System-Thread ausgelagert.

Zunächst wird das multilinguale Transformer-Modell `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`¹² in Kombination mit der `KeyBERT`-Bibliothek¹³ (als `Middleman`) geladen.

Die Textinhalte werden über die Funktion `clean()` von Zeilenumbrüchen und Tabs/Lehrzeichen befreit. `KeyBERT` extrahiert anschließend N-Gramme (1 bis 2 Wörter) unter Anwendung von Maximal Marginal Relevance (MMR) mit einer Diversität von 0.7. Um den Modulnamen mit einzubeziehen, da dieser viel Kontext über die Modulinhalte und Orientierung gibt, wird ein Vektor aus dem Lehrinhalt erstellt und mit dem Vektor des Modulnamens gemittelt, wodurch ein ausgeglichener "Vibe-Vektor" entsteht.

Die finalen Schlagwörter (`data["words"]`) entstehen aus einer Kombination von zwei Quellen:

- **Interne Text-Keyphrases (KeyBERT):** `KeyBERT` extrahiert N-Gramme (als Unigramme und Bigramme im Bereich von 1 bis 2 Wörtern) direkt aus dem Lehrinhalt. Dabei wird Maximal Marginal Relevance (MMR) mit einer Diversität von 0.7 angewandt, um redundante Wortwiederholungen zu minimieren. Die Top-3-Ergebnisse hieraus bilden den ersten Teil der Keywords.

¹¹siehe [PyTorch]

¹²siehe [SBERT]

¹³siehe [KeyBERT]

- **Externe Fachdisziplinen (`sparql_candidates`):** Um den fachlichen Kontext zu erweitern, wird ein „Vibe-Vektor“ gebildet (durch mathematische Mittelung der Vektoren aus Lehrinhalt und Modulname). Dieser wird mittels `util.cos_sim` mit einer vorab encodierten Embedding-Matrix akademischer Disziplinen abgeglichen. Die Disziplinen werden über die Hilfsfunktion `sparql_candidates()` aus einer lokal gecachten JSON-Datei (`academic_disciplines.json`) eingelesen, welche ursprünglich über eine Wikidata-SPARQL-Query generiert wurde.

Mit `torch.topk` werden die passendsten Ergebnisse gefiltert, wenn der Cosine-Score größer als 0.4 ist. Das Ergebnis sind bis zu 6 genaue, semantisch passende Schlagwörter pro Modul. Diese werden direkt im Daten-Dictionary gespeichert. Diese werden bei jedem Dictionary-Eintrag zur jeweiligen Modul-ID hinzugefügt und schließlich auf Supabase in ein Bucket hochgeladen.

4.4 Backend bei TypeScript

Alessio Beulcke

Die anwendungsnahe Server-Logik wird direkt in TypeScript über Next.js Server Actions umgesetzt. Diese bilden die Vermittlungsschicht zwischen den React-Komponenten des Frontends (Abschnitt 3) und der Supabase-Datenbank (Abschnitt 5), indem sie sämtliche Lese- und Schreibzugriffe auf die relationalen Tabellen kapseln und dem Client ausschließlich aufbereitete Ergebnisse bereitstellen.

Server Actions werden über die Direktive `use server` ausgezeichnet und ausschließlich serverseitig ausgeführt. Dies erfolgt, weil zum einen die Zugangsdaten für die MOSES-API (`MOSES_API_KEY`) den Server nicht verlassen dürfen, weshalb API-Aufrufe nicht clientseitig erfolgen, und zum anderen werden dadurch CORS-Restriktionen umgangen, da die Anfragen server-to-server statt aus dem Browser gestellt werden.

Die zentrale Funktion `getSemestersMitModulen` in `planner/actions.ts` lädt die Semester eines Nutzers samt der zugeordneten Module. Da die Zuordnung von Modulen zu Semestern über eine separate Verknüpfungstabelle (`planner`) erfolgt, was das Verschieben von Modulen zwischen Semestern ermöglicht, lädt die Funktion in der Reihenfolge `semester → planner → module`. Die pro Semester notwendigen Modulabfragen werden über `Promise.all` gebündelt, um die Latenz gering zu halten. Anschließend werden die Datenbankfelder auf den im Frontend erwarteten Typ gemappt.

Für die Interaktion des Planers stehen dedizierte Server Actions bereit:

- `verschiebeModul` aktualisiert bei einem Drag-and-Drop-Vorgang die Semesterzuordnung eines Moduls, indem der Fremdschlüssel `group_id` in der `planner`-Tabelle neu gesetzt wird.
- `loescheModul` und `loescheSemesterMitModulen` kapseln die Löschlogik; die Verknüpfungen der Tabellen auf Supabase werden über `on delete cascade` auf Datenbankebene automatisch entfernt.
- `moduleZuPlanerHinzufuegen` legt eine neue Verknüpfung an und fängt den PostgreSQL-Fehlercode 23505¹⁴ (Verletzung des Unique-Constraints `unique_user_moses_modul`) ab, um doppelte Einträge kontrolliert zu behandeln.
- `findeModulImPlaner` prüft, ob ein Modul bereits im Planer vorhanden ist und liefert neben dem booleschen Status auch das zugehörige Semester zurück.

Die Funktionen `createSemester` und `deleteSemester` halten die in den Einstellungen hinterlegte Studiendauer (`max_semester`) und das aktuelle Semester (`current_semester`) mit dem tatsächlichen Zustand des Planers konsistent. So zieht das Anlegen bzw. Löschen von Semestern im Planer den Maximalwert nach, während das aktuelle Semester bei Bedarf mitgeführt wird, sodass es die Studiendauer nie überschreitet.

Von den datenbanknahen Funktionen entkoppelt liegt die reine Berechnungslogik in `lib/utils.ts`. Hierzu zählen `berechneTurnus`, das den Turnus eines Semesters per Paritätslogik aus dem aktuellen Semester ableitet, sowie `handleModule`, das die uneinheitlichen Formate einer Modul-ID (Objekt bzw. String/Number) auf einen einheitlichen String normalisiert.

4.5 Automation

Alessio Beulcke

NAVIS enthält eine automatisierte Hintergrundlogik zur Fortschreibung des aktuellen Semesters. Das im Nutzerprofil (`profiles`) hinterlegte `current_semester` wird zu den Semesterstartterminen (01.04 und 01.10) eigenständig um die Anzahl der zwischenzeitlich vergangenen Semester erhöht.

Da die App auf Vercel serverless betrieben wird, existiert kein dauerhafter Prozess für einen zeitgesteuerten Stichtag. Statt eines Cron-Jobs wird ein **Compute-on-Load** Ansatz verwendet. Das gemeinsame Layout `app/protected/layout.tsx` (Server-Komponente) ruft beim Rendern die Server Action `pruefeSemesterUpdate()` auf, wodurch die Prüfung bei jedem Aufruf einer geschützten Seite greift. Die Ermittlung des aktuellen Datums

¹⁴siehe [PGErrorCodes]

`new_Date()` erfolgt serverseitig und ist damit unabhängig von der Systemzeit des Endgeräts vom Nutzer.

Um die Anzahl zwischen zwei Daten liegenden Semesterstarttermine ohne Fallunterscheidung zu bestimmen, zählt die Hilfsfunktion `wechselBisDatum` in `lib/utlis.ts` die bis zu einem Datum insgesamt vergangenen Wechsel. Bei zwei Wechseln pro Jahr ergibt sich:

$$\text{wechselBisDatum}(\text{datum}) = (\text{Jahr} \cdot 2) + (\text{Datum} \geq 01.04. ? 1 : 0) + (\text{Datum} \geq 01.10. ? 1 : 0)$$

Die tatsächliche Anzahl der Semesterübergänge zwischen dem vorherigen Kontrolldatum und dem aktuellen Datum ergibt sich als Differenz zweier solcher Zählstände und ist damit für alle Zeiträume identisch definiert.

Beispielrechnung: Ein Nutzer mit dem Kontrolldatum `last_semester_update = 10.04.2025` ruft die App am 20.12.2026 erneut auf. Für beide Daten wird der Zählstand bestimmt:

$$\text{wechselBisDatum}(20.12.2026) = (2026 \cdot 2) + 1 + 1 = 4054$$

$$\text{wechselBisDatum}(10.04.2025) = (2025 \cdot 2) + 1 + 0 = 4051$$

$$\text{Semesterübergänge} = 4054 - 4051 = 3$$

Kontrolle: Semesterübergänge vom Kontrolldatum (10.04.2025) bis zum aktuellen Datum (20.12.2026):

- 1. Übergang am 01.10.2025
- 2. Übergang am 01.04.2026
- 3. Übergang am 01.10.2026

Dies bestätigt die berechnete Differenz von 3.

Die Spalte `last_semester_update` (Typ `date`) unter der Supabase Tabelle `profiles` dient als Zeitanker/Kontrolldatum der letzten erfolgten automatisierten Semesterübergänge. Die Funktion `pruefeSemesterUpdate()` unterscheidet:

- **Anker ist NULL:** Anker auf heute setzen, keine Erhöhung (kein Bezugspunkt).

- **Differenz 0:** keine Schreiboperation.
- **Differenz > 0:** Semester erhöhen, Anker auf heute fortschreiben.

Die Erhöhung wird über `Math.min(current + anzahl, max_semester)` gegen die maximale Studiendauer gedeckelt. Da sich Winter- und Sommersemester zwangsläufig abwechseln, wird zusätzlich `current_turnus` über `berechneTurnus` auf Basis des neuen Semesters neu bestimmt (Paritätslogik: ungerade Übergangszahlen kippt den Turnus, gerade erhält ihn).

Das gesamte Automationsfeature ist als Opt-in angelegt. Die Spalte `auto_semester_update_enabled` (Typ `boolean`, Standard `false`) in der `profiles` Tabelle unter Supabase wird über eine Checkbox in `update-semester-form.tsx` gesteuert. Bei deaktivierter Automation wird der Anker weiterhin auf das aktuelle Datum des Login fortgeschrieben, das `current_semester` wird unverändert beibehalten. Dies verhindert, dass ein späteres Aktivieren die zwischenzeitlich vergangenen Semesterübergänge nachträgt.

5 Supabase

Anna Glavatska

Supabase (BaaS Architektur) dient in NAVIS als zentrale Backend- und Datenbank-Infrastruktur auf Basis von PostgreSQL¹⁵.



Abbildung 2: Supabase Tabellen Visualisierung

5.1 Buckets

Für die Verwaltung unstrukturierter und binärer Daten setzt NAVIS auf Supabase Storage in Form von Buckets.

Im Bereich der Nutzerprofile werden die Profilbilder der Studierenden im Bucket **avatars** unter dem nutzerspezifischen Pfad `${userId}/avatar.png` isoliert hinterlegt. Der Datei-Upload erfolgt dabei mittels einer Upsert-Logik direkt über die Web-App, woraufhin die öffentliche URL (`avatar_url`) inklusive eines Cache-Busting-Timestamps in der Tabelle **profiles** aktualisiert wird. Ebenso wird das physische Löschen eines Bildes aus dem Storage synchron mit dem Zurücksetzen des entsprechenden Datenbankeintrags auf NULL gehandhabt.

Neben der reinen Bildverwaltung dienen die Buckets zudem als Zwischen- und Endspeicher für größere JSON-Strukturen innerhalb der Datenverarbeitung. Die in der asynchronen Python-Pipeline generierten Inhalte werden als strukturierte JSON-Dateien direkt in ein Storage-Bucket hochgeladen.

¹⁵siehe [PostgreSQL]

Diese gezielte Auslagerung entlastet den Datenbank-Traffic bei komplexen Suchanfragen nachhaltig und gewährleistet eine hohe Performance der Anwendung.

Die Erstellung neuer Konten (`SignUpForm` über `app/auth/sign-up`) sowie der Passwort-Reset (`ForgotPasswordForm` in `app/auth/forgot-password` und `UpdatePasswordForm` in `app/auth/update-password`) nutzen tokenbasierte E-Mail-Links beziehungsweise Einmalpasswörter (One-Time Password, kurz OTP).

Über `supabase.auth.resetPasswordForEmail()` fordert der Client im Passwort-Reset-Flow einen passwortgeschützten Reset-Link an, der den Nutzer dynamisch auf die Route `/auth/update-password` weiterleitet (`window.location.origin`). Die dort eingebundene `<UpdatePasswordForm/>` ermöglicht anschließend das sichere Setzen neuer Zugangsdaten innerhalb der authentifizierten Session. Da während der Entwicklungs- und Testphase restriktive Versandlimits bei universitären SMTP-Servern (< 200 Mails/Minute) sowie im kostenlosen Test-Tier (2 Mails/Stunde) auftraten, wurde die Architektur um eine custom Resend-OTP-Logik und die Anbindung eines nativen SMTP-Servers erweitert. Dies verhindert, dass Studierende bei wiederholten Registrierungs- oder Reset-Anfragen durch Sicherheitsmechanismen blockiert werden oder inkonsistente „Ghost-Nutzer“ in der Datenbank entstehen. Zusätzlich sichert ein defensives UI-Design sämtliche Auth-Formulare durch asynchrone Ladezustände (`isLoading`) und deaktivierte Schaltflächen (`disabled={isLoading}`) ab, um parallele Mehrfachanfragen während laufender Netzwerkopoperationen zu blockieren.

Um zu verhindern, dass Nutzer auf fremde Daten zugreifen können, ist auf allen relationalen Datenbanktabellen (`profiles`, `module`, `semester`, `planner`, `bewertung`, `logs`) Row Level Security (RLS) aktiviert.

Während Supabase sensible Identitätsdaten wie E-Mail-Adressen und Passwort-Hashes isoliert in `auth.users` verwaltet, speichert die öffentliche Tabelle `profiles` anwendungsbezogene Metadaten. Ihr Primärschlüssel `profiles.id` ist als Fremdschlüssel 1:1 an `auth.users.id` gebunden und pflegt studiengangsspezifische Attribute (siehe Abb. 3) wie `studiengang` (Anzeigename des Studiengangs), `studiengang_id` (Zugehörige ID des Studiengangs für die Moses-API-Calls), `current_semester` (Aktuelles Semester des Nutzers), `max_semester` (Studiendauer des Nutzers), `current_turnus` (Aktueller Turnus des Nutzers), `stupo_year` (StuPo-Jahr des gewählten Studiengangs), `last_semester_update` (Eintrag des letzten erfolgreich ausgeführten Kontrolldatum), `total_ects` (Eingegebene Max Anzahl der zu erreichenden ECTS Punkte für die gesamte Studiendauer), `target_grade` (Wunsch des Abschlussnotendurchschnitts), `auto_semester_update_enabled` (Wahrheitsstand für die/den Checkbox/Toggle des automatisierten Semesterübergangs) sowie administrative Rechte (`is_admin`). Jede SQL-Operation (`SELECT`, `INSERT`, `UPDATE`, `DELETE`)

wird datenbankseitig durch sitzungsbasierte RLS-Policies gegen den Kontext des authentifizierten Nutzers geprüft.

Abfragen sind nur zulässig, wenn die Identitätsspalte des Datensatzes mit der User-ID des aktiven JSON Web Tokens (`user_id = auth.uid()`) übereinstimmt.

Administrative Eingriffe, wie beispielsweise das Triggern der asynchronen Python-ETL-Pipeline oder das Lesen der Systemlogs (`logs`-Tabelle), erfordern ein explizites Rollenprivileg über die boolesche Spalte `is_admin`. Nicht-administrative Nutzer werden durch einen Admin-Guard beim Aufruf des Admin-Bereichs abgewiesen und umgeleitet.

5.2 Authentifizierung und Sicherheit

profiles		
 	id	uuid
 	username	text
	studiengang	text
	avatar_url	text
	studiengang_id	int4
	is_admin	bool
	current_semester	int4
	max_semester	int4
	current_turnus	text
	stupo_year	text
	last_semester_update	date
	total_ects	int4
	target_grade	numeric
	auto_semester_update_e...	bool

Abbildung 3: Tabelle zum Nutzer-Profil

Standardmäßig erfordert die Methode `signInWithPassword` von Supabase Auth eine gültige E-Mail-Adresse und ein Passwort. Um den Studierenden eine flexiblere Nutzerführung zu bieten, unterstützt die Anmeldemaske (Login-Form in `app/auth/login`) optional den Login über den individuell vergebenen Benutzernamen (`username`). Da die interne Authentifizierungstabelle `auth.users` von Supabase Benutzernamen nicht primär für Login-Abfragen exponiert, wurde hierfür ein mehrstufiger Ablauf implementiert.

Beim Absenden des Anmeldeformulars prüft der Client zunächst durch eine Eingabeanalyse, ob der eingegebene Identifier ein `@`-Zeichen enthält (`identifier.includes("@")`). Handelt es sich um einen Benutzernamen, wird die serverseitige PostgreSQL-Funktion `get_email_by_username` über `supabase.rpc()` aufgerufen, welche datenbankseitig abläuft und den Benutzernamen sicher in die zugehörige E-Mail-Adresse auflöst.

Erst mit der so ermittelten E-Mail-Adresse schlägt der Client den eigentlichen Authentifizierungsrequest (`signInWithPassword`) an Supabase Auth an. Ein fehlertolerantes Error-Handling übersetzt dabei generische API-Rückmeldungen wie `Invalid login credentials` in benutzerfreundliche, deutschsprachige Fehlermeldungen (z. B. "Passwort ist falsch." oder "Benutzername nicht gefunden."). Nach erfolgreicher Validierung wird der Nutzer direkt auf die geschützte Route `/protected/settings` weitergeleitet.

5.3 Planer und Relationale Tabellen

Alessio Beulcke

Die anwendungsbezogenen Daten des Studienplaners verteilen sich auf mehrere relationale Tabellen, die über Fremdschlüssel an den authentifizierten Nutzer (`auth.user.id`) und untereinander gebunden sind (siehe Abb. 2 und Abb. 4)

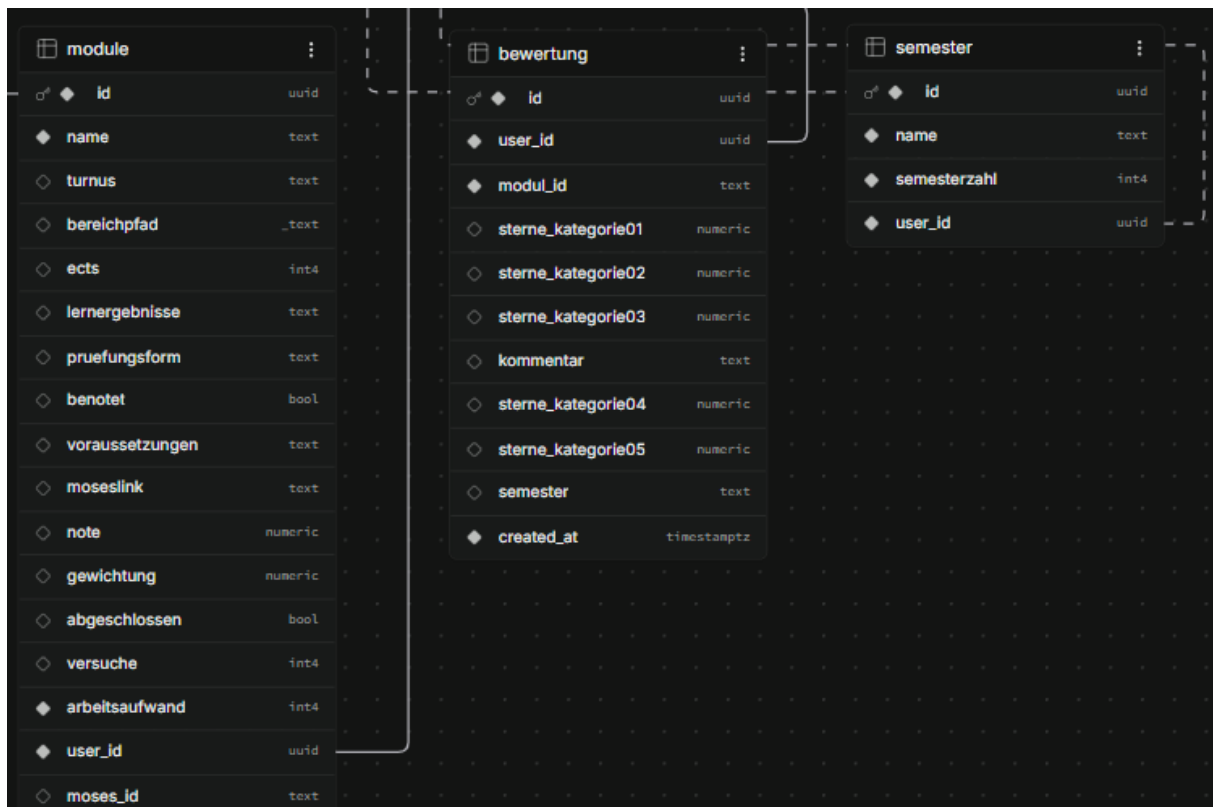


Abbildung 4: Tabellen: Modulinformation, Nutzerbewertung und Semester des Nutzers

Die Tabelle **module** bildet den zentralen Datenspeicher des Planers und hält die Moduldaten nutzerspezifisch. Jeder Eintrag ist über `user_id` an genau einen Nutzer gebunden. Die Attribute lassen sich in drei Gruppen einteilen:

- **Fachliche Moduldaten:** `name` und `ects` beschreiben das Modul grundlegend. `turnus` gibt den Semesterrhythmus an (Winter-, Sommer- oder beide Semester), `bereichpfad` ordnet das Modul einem Studienbereich zu (etwa Pflicht oder Wahlpflicht). `pruefungsform`, `lernergebnisse` und `voraussetzungen` übernehmen die inhaltlichen Angaben aus MOSES, `arbeitsaufwand` den geschätzten Zeitaufwand.
- **Studentische Leistungsdaten:** Diese Felder pflegt der Nutzer individuell. `note` und `gewichtung` dienen der Notenberechnung, `versuche` zählt die Prüfungsversuche, `abgeschlossen` und `benotet` kennzeichnen den Status des Moduls.

- **MOSES-Bezug:** Über `moses_id` bleibt die Verbindung zum ursprünglichen MOSES-Modul erhalten, `moseslink` verweist auf die Modulbeschreibung auf der MOSES-Webseite der TU Berlin.

Die Tabelle `semester` bildet die einzelnen Fachsemester eines Nutzers ab und hält je Eintrag einen `name` (Anzeigenname des Semesters) sowie eine `semesterzahl` (numerischer Wert des Fachsemesters) und ist ebenfalls über die `user_id` an den Nutzer gebunden.

Die Tabelle `bewertung` hinterlegt Nutzerbewertungen zu Modulen. Neben den zugehörigen Feldern `user_id` und `modul_id` werden fünf Bewertungskategorien (`sterne_kategorie01` bis `sterne_kategorie05`) als numerische Werte für die abgegebene Sternbewertung, ein Freitextkommentar in `kommentar` sowie ein Zeitstempel (`created_at`) gespeichert.



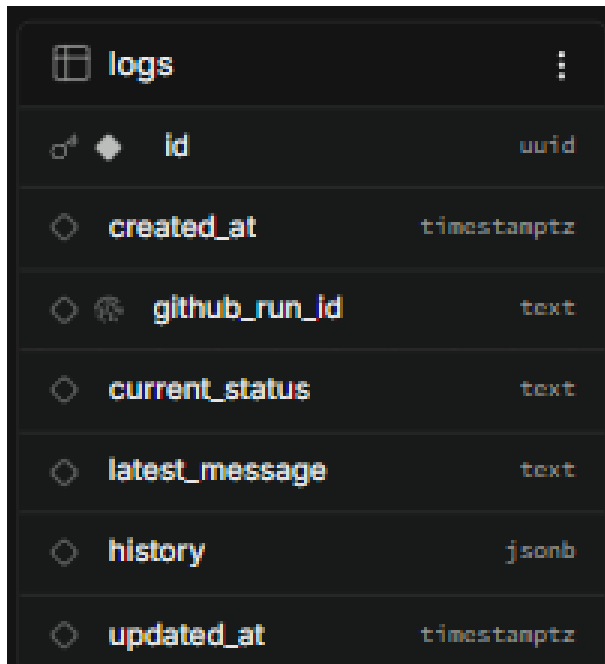
planner	
id	uuid
group_id	uuid
user_id	uuid
modul_id	uuid

Abbildung 5: Assoziative Tabelle

Die Zuordnung von Modulen zu Semestern erfolgt nicht direkt, sondern über die assoziative Tabelle `planner` (siehe Abb. 5). Diese hält je Eintrag die drei Fremdschlüssel `modul_id` (Module), `group_id` (Semester) und `user_id` (Nutzer) und entkoppelt so die Module von den Semestern. Erst diese Entkopplung ermöglicht das freie Verschieben eines Moduls zwischen Semestern per Drag-and-Drop, ohne die eigentlichen Moduldaten zu verändern (verschoben wird lediglich der `group_id` Verweis). Die Fremdschlüssel sind mit `ON DELETE CASCADE` definiert, sodass beim Löschen eines Semesters oder Moduls die zugehörigen Planer-Verknüpfungen automatisch entfernt werden und keine verwaisten Einträge zurückbleiben.

5.4 Debugging und Realtime Logging

Anna Glavatska



The screenshot shows the Supabase Admin interface for a table named 'logs'. The table has the following columns and data types:

id	uuid
created_at	timestampz
github_run_id	text
current_status	text
latest_message	text
history	jsonb
updated_at	timestampz

Abbildung 6: Logs für Debugging und Admin-Aktionen

Das Realtime-Logging ermöglicht die Ablaufsübersicht und das Debugging der asynchronen Python-ETL-Pipeline direkt über die NAVIS-Admin-Page. Während der Ausführung übermittelt das Python-Skript Statusupdates (`PROCESSING`, `SUCCESS`, `WARNING`, `ERROR`) sowie detaillierte Prozessmeldungen via `send_status_admin()` per Upsert an die Supabase-Tabelle `logs`. Die Daten werden dort strukturgebunden mit dem eindeutigen Bezeichner `github_run_id` für Upserts sowie der Ausführungshistorie im `jsonb`-Format hinterlegt. Um die Performance der Pipeline nicht zu beeinträchtigen, wird der I/O-Schreibvorgang mittels `asyncio.to_thread` entkoppelt. Im Frontend lauscht ein Supabase-Realtime-Listener per WebSocket direkt auf Tabellenänderungen, was den Prozessstatus ohne ständiges HTTP-Polling oder Seiten-Reloads ressourcenschonend in Echtzeit visualisiert.

Außerdem wurde ein Workflow `auto_keep_alive.yml` hinzugefügt, der alle 3 Wochentage die neue Tabelle `auto_ping` abfragt ("status") und somit als Fallback bei geringer Nutzeraktivität (z.B.) während der vorlesungsfreien Zeit dient, um das automatisierte Pausieren der Datenbank zu verhindern. Falls die Verbindung zur Datenbank fehlschlägt, werden der Statuscode 1 und eine Anweisung zurückgegeben, das Dashboard von Supabase zu besuchen und die Aktivität fortzusetzen. (-> Dashboard -> Unpause)

6 Vercel

6.1 Hosting: Next.js App auf Vercel

Anna Glavatska

Vercel dient als primäre Cloud-Hosting- und Deployment-Plattform für unsere NAVIS Web App. Über Vercel wird das gesamte Frontend sowie die Server-Logik (Server Components und Server Actions) bereitgestellt.

Durch die direkte Integration mit dem GitHub-Repository¹⁶ stellt Vercel eine automatische CI/CD-Pipeline bereit: Jeder Code-Push löst direkt ein neues Build-Deployment aus, während Pull-Requests in isolierten Preview-Umgebungen getestet werden können. Zudem ermöglicht Vercel die geschützte Verwaltung von Umgebungsvariablen (wie API-Schlüsseln für Supabase und MOSES).

6.2 Serverless Vercel

Serverless Architecture (auch Function-as-a-Service, FaaS) bedeutet, dass Servereinrichtung, Wartung oder Skalierung automatisch von Vercel durchgeführt wird. Statt eines dauerhaften physischen Server, wird der Code erst bei einer eingehenden Nutzeranfrage in einem isolierten, virtuellen Container kurzzeitig gestartet, führt die gewünschte Aktion aus und schaltet sich danach sofort wieder ab. Das garantiert eine automatische Skalierung bei vielen Zugriffen, bringt jedoch strikte Zeit- und Ressourcenlimits pro Anfrage mit sich (wie das 10-Sekunden-Timeout auf Vercel), weshalb rechenintensive Aufgaben entkoppelt werden mussten.

6.3 Progressive Web App

Für die zuverlässige Nutzbarkeit als Progressive Web App (PWA) wurde die Basis-Konfiguration auf den aktuellen Vercel-Standard gebracht. Zusätzlich wurden in der `next.config.ts` sowie in der `manifest.json` die PWA-Grundlagen und Pakete conditional auf Deployment (`npm run dev`) und Production (`npm run build`) und das Proxy-Routing angepasst, um das fehlerhafte Abfangen des Service Workers (`sw.js`), der Workbox-Instrumente und des Manifests zu verhindern.

Dazu wurden in der `package.json` die Build-Skripte (Webpack für Production-Builds und Turbopack für Deployments) eingerichtet sowie Anpassungen in der `layout.ts`

¹⁶siehe [GitHub]

vorgenommen, was zusammen eine verlässliche Installation und saubere Updates der App auf Mobilgeräten garantiert.

7 Git

Anna Glavatska

7.1 Workflows und CI/CD

- **GitHub Actions für die erweiterte Suche:** Zur Automatisierung des isolierten Python-Skripts wurde ein spezifischer Workflow (`update-extended-search.yml`) als CI/CD-Pipeline eingerichtet.
- **Trigger-Logik:** Die Pipeline unterstützt drei Ausführungswege: Durch ein externes Event (`repository_dispatch`), manuell durch einen Admin über die NAVIS Web App oder automatisiert via Cron-Job monatlich am Ersten um 03:00 Uhr (`workflow_dispatch`).
- **Security und Container-Build:** Innerhalb eines `ubuntu-latest` Runners wird das Repository ausgecheckt und das Docker-Image gebaut. Sensible Daten (wie `MOSES_API_KEY` und `SERVICE_ROLE_KEY`) werden dabei sicher aus dem GitHub Secrets Vault ausgelesen und als Umgebungsvariablen direkt in den laufenden Container übergeben.

7.2 DevOps und Branchmanagement

Für die Versionskontrolle und das Code-Management kam in unserem Team ein strukturierter Feature-Branch-Workflow zum Einsatz. Der Haupt-Branch (`main`) repräsentierte dabei fortlaufend den stabilen Produktionszustand der Anwendung. Um ungetesteten Code oder versehentliche Änderungen am Live-System zu verhindern, wurden strikte **Branch-Protection-Rules** auf GitHub konfiguriert: Direkte Pushes auf den Haupt-Branch waren technisch blockiert, sodass Änderungen ausschließlich über Pull Requests (PRs) nach einer expliziten Freigabe (Approval) durch weitere Teammitglieder integriert werden konnten. Administrative Rechte für Repository-Einstellungen, Secrets und Pipeline-Konfigurationen waren dabei fest definierten Admin-Rollen vorbehalten.

Unterstützt wurde dieser Prozess durch Vercels automatische **Preview-Deployments**: Jede Änderung ließ sich vorab in einer isolierten Live-Umgebung sowohl auf funktionale Korrektheit als auch auf responsives UI-Verhalten testen, bevor sie in den Haupt-Branch einfluss.

Neben den rein technischen DevOps-Mechanismen stellte die direkte Teamkommunikation außerhalb von GitHub einen entscheidenden Erfolgsfaktor für das Projekt dar. Durch

kontinuierliche (virtuelle) Meetings und den schnellen Austausch über Team-Chats konnten architektonische Entscheidungen effizient abgestimmt, Merge-Konflikte proaktiv vermieden und ein gemeinsames Verständnis für die gesamte Codebasis geschaffen werden.

7.3 Ausblick

Git-Basis

Das bestehende GitHub-Repository wird übernommen oder geforkt, sodass die aktuellen ETL-Pipelines, Skripte und GitHub Actions ohne Anpassungen weiterlaufen können – vorausgesetzt, die Zugriffsrechte werden entsprechend eingeschränkt. Da die App auf Supabase aufbaut, wird dringend empfohlen, dies beizubehalten.

Umzug auf einen TU-Server

Das System wird auf eine virtuelle Maschine (VM) der TU migriert, was allerdings eine Anpassung an die TU-Architektur und eine Neueinrichtung der Trigger erfordert.

Bei beiden Grundoptionen ist es vorteilhaft bzw. empfohlen, Supabase beizubehalten. Alle weiteren Dateien, die keinen Bezug zu GitHub-Actions haben, kann man in beiden Fällen nur auf den TU-Servern lassen.

Feed-Funktion

Hinzufügen eines Feeds, bei dem ein Admin über die Admin-Seite Updates und Informationen posten kann, welche dann dem Nutzer angezeigt werden (z. B. bei neuen Funktionen oder Wartungsarbeiten).

Abbildungsverzeichnis

1	Farbschema erstellt mit coolers.	10
2	Supabase Tabellen Visualisierung	24
3	Tabelle zum Nutzer-Profil	27
4	Tabellen: Modulinformation, Nutzerbewertung und Semester des Nutzers .	28
5	Assoziative Tabelle	29
6	Logs für Debugging und Admin-Aktionen	30

Codelistenverzeichnis

1	Struktur des Modul-Dictionarys in der ETL-Pipeline	19
---	--	----

Quellenverzeichnis

- [Supabase] Supabase, Inc.: *Supabase*, 2026.
<https://supabase.com>
Zugriff: 06.08.2026, 13:00 Uhr
- [Nextjs] Vercel: *Next.js*, 2026.
<https://nextjs.org>
Zugriff: 05.08.2026, 16:19 Uhr
- [React] Meta Platforms, Inc.: *React*, 2026.
<https://react.dev>
Zugriff: 06.08.2026, 12:59 Uhr
- [TypeScript] Microsoft: *TypeScript*, 2026.
<https://www.typescriptlang.org>
Zugriff: 06.08.2026, 00:51 Uhr
- [Vercel] Vercel, Inc.: *Vercel*, 2026.
<https://vercel.com>
Zugriff: 06.08.2026, 13:02 Uhr
- [TailwindCSS] Tailwind Labs, Inc.: *Tailwind CSS*, 2026.
<https://tailwindcss.com>
Zugriff: 06.08.2026, 13:00 Uhr
- [MOSES] Technische Universität Berlin: *MOSES – Modultransfersystem*, 2026.
<https://moseskonto.tu-berlin.de/moses/>
Zugriff: 06.08.2026, 13:01 Uhr
- [dnd-kit] Claudéric Demers: *dnd kit*, 2026.
<https://dndkit.com>
Zugriff: 06.08.2026, 13:05 Uhr
- [Docker] Docker, Inc.: *Docker*, 2026.
<https://www.docker.com>
Zugriff: 06.08.2026, 13:06 Uhr
- [GitHubActions] GitHub, Inc.: *GitHub Actions*, 2026.
<https://github.com/features/actions>
Zugriff: 06.08.2026, 13:11 Uhr
- [PyTorch] PyTorch Foundation (The Linux Foundation): *PyTorch*, 2026.
<https://pytorch.org>
Zugriff: 06.08.2026, 13:07 Uhr

- [KeyBERT] Maarten Grootendorst: *KeyBERT*, 2026.
<https://maartengr.github.io/KeyBERT/>
Zugriff: 06.08.2026, 13:07 Uhr
- [SBERT] UKP Lab / Hugging Face: *Sentence Transformers*, 2026.
<https://www.sbert.net>
Zugriff: 06.08.2026, 13:08 Uhr
- [PGErrorCodes] The PostgreSQL Global Development Group: *PostgreSQL Error Codes*, 2026.
<https://www.postgresql.org/docs/current/errcodes-appendix.html>
Zugriff: 06.08.2026, 00:57 Uhr
- [PostgreSQL] The PostgreSQL Global Development Group: *PostgreSQL*, 2026.
<https://www.postgresql.org>
Zugriff: 06.08.2026, 13:08 Uhr
- [GitHub] GitHub, Inc.: *GitHub*, 2026.
<https://github.com/TuBerlin-Projekt-Medienerstellung/StudiKompass>
Zugriff: 06.08.2026, 13:12 Uhr
- [Git] Software Freedom Conservancy (Git-Projekt): *Git*, 2026.
<https://git-scm.com>
Zugriff: 06.08.2026, 13:11 Uhr
- [coolers] color palettes generator, *Coolors srl*
<https://coolors.co/>
Zugriff: 06.08.2026, 20:32 Uhr